

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №6
по дисциплине
«Функциональное программирование»
Вариант 47

Студент,
группы 5130201/30102

_____ Санько В. В.

Преподаватель

_____ Моторин Д. Е.

« _____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
Постановка задачи	4
1 Реализация	5
1.1 Библиотека функций (Lib.hs)	5
1.2 Тестирование на основе свойств (Spec.hs)	5
1.3 Свойства глубины дерева	6
2 Тестирование	7
Заключение	9
Список литературы	10

Введение

В функциональном программировании, в частности в языке Haskell, широкое распространение получил подход Property-based testing (тестирование на основе свойств). В отличие от классического модульного тестирования, где программист вручную задает входные данные, данный подход позволяет описывать общие свойства функций, которые должны выполняться для любых допустимых входных значений.

В данной работе рассматривается использование библиотеки **QuickCheck** для автоматической генерации тестовых данных и проверки математических свойств функций сравнения по модулю и вычисления глубины бинарного дерева. Проект организован с использованием инструмента сборки **stack**, что обеспечивает модульность и корректное управление зависимостями.

Постановка задачи

Цель работы: Реализовать библиотеку для работы с модульной арифметикой и бинарными деревьями, обеспечив покрытие кода автоматизированными тестами на основе свойств.

Задачи:

- Создать проект с использованием сборщика `stack`, выделив логику в модуль `Lib.hs`, а тесты — в `Spec.hs`.
- Реализовать функцию `isCongruent` для проверки равенства чисел по модулю и ограничить доступ к вспомогательным функциям.
- Проверить свойства `isCongruent`: связь с разностью чисел, симметричность и рефлексивность.
- Реализовать структуру данных `Tree` и функцию `treeDepth`.
- Описать экземпляр класса `Arbitrary` для автоматической генерации случайных бинарных деревьев.
- Проверить свойства глубины дерева: значения для пустого и единичного дерева, рекуррентное соотношение и монотонность при добавлении узлов.

1 Реализация

1.1 Библиотека функций (Lib.hs)

В модуле `Lib` реализована основная логика программы. Для обеспечения инкапсуляции вспомогательная функция `modulus` не включена в список экспорта модуля.

```
1  module Lib
2    ( isCongruent
3    , Tree(..)
4    , treeDepth
5    ) where
6
7    modulus :: Int -> Int -> Int
8    modulus a b
9      | b == 0    = 0
10     | otherwise = a `mod` abs b
11
12    isCongruent :: Int -> Int -> Int -> Bool
13    isCongruent a b m
14      | m == 0    = error "0"
15     | otherwise = (a `mod` m) == (b `mod` m)
16
17    data Tree a = Empty | Node a (Tree a) (Tree a)
18    deriving (Show, Eq)
19
20    treeDepth :: Tree a -> Int
21    treeDepth Empty = 0
22    treeDepth (Node _ left right) = 1 + max (treeDepth left) (treeDepth right)
23
```

Представленный программный код обеспечивает базовый функционал, необходимый для выполнения математических операций и манипуляций со структурами данных, сохраняя при этом чистоту интерфейса модуля.

1.2 Тестирование на основе свойств (Spec.hs)

Модуль тестов использует библиотеку `QuickCheck`. Для тестирования деревьев определен генератор случайных структур с контролем размера (`sized`), что предотвращает бесконечную рекурсию при генерации.

```
1  {-# LANGUAGE ScopedTypeVariables #-}
2  import Lib
3  import Test.QuickCheck
4  import Control.Exception (evaluate, try, SomeException)
5
6  prop_congruentDifference a b (NonZero m) =
7    isCongruent a b m == ((a - b) `mod` m == 0)
8
9  prop_congruentSymmetric a b (NonZero m) =
10    isCongruent a b m == isCongruent b a m
11
12  prop_congruentEqual a (NonZero m) =
13    isCongruent a a m == True
14
15  prop_zeroModulusThrows a b = ioProperty $ do
16    result <- try (evaluate (isCongruent a b 0))
```

```

17     return $ case result of
18     Left (_ :: SomeException) -> True
19     Right _                    -> False
20
21     instance Arbitrary a => Arbitrary (Tree a) where
22     arbitrary = sized genTree
23     where
24     genTree 0 = return Empty
25     genTree n = do
26     val <- arbitrary
27     left <- genTree (n `div` 2)
28     right <- genTree (n `div` 2)
29     return $ Node val left right
30

```

Использование специализированных типов, таких как `NonZero`, и механизмов перехвата исключений позволяет библиотеке `QuickCheck` проводить стресс-тестирование функций даже в условиях потенциально некорректных входных данных.

1.3 Свойства глубины дерева

Для проверки работы с деревьями реализованы свойства, описывающие базовые случаи и поведение структуры при изменениях.

```

1     prop_emptyTreeDepth = treeDepth (Empty :: Tree Int) == 0
2
3     prop_singleNodeDepth val = treeDepth (Node val Empty Empty) == 1
4
5     prop_treeDepthMax val l r =
6     treeDepth (Node val l r) == 1 + max (treeDepth l) (treeDepth r)
7
8     prop_addNodeNotDecrease t v = treeDepth (addNode v t) >= treeDepth t
9     where
10    addNode x Empty = Node x Empty Empty
11    addNode x (Node val l r) = Node val (addNode x l) r
12

```

Данный набор свойств формирует полную спецификацию ожидаемого поведения функции `treeDepth`, что позволяет гарантировать стабильность работы алгоритма при изменении внутренней реализации деревьев.

2 Тестирование

Запуск тестов осуществляется командой `stack test`. Система автоматически генерирует по 100 наборов входных данных для каждого свойства.

Пример вывода результатов тестирования:

```
--- Testing isCongruent ---
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.

--- Testing treeDepth ---
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
```

Рис. 1: Результаты выполнения тестов QuickCheck

```
--- Testing isCongruent ---
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
*** Failed! Falsified (after 1 test and 1 shrink):
0
NonZero {getNonZero = 1}
+++ OK, passed 100 tests.

--- Testing treeDepth ---
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
```

Рис. 2: Вызов ошибок в первой части

В ходе тестирования были проверены следующие сценарии:

1. **isCongruentSymmetric**: Подтверждена симметричность отношения эквивалентности.
2. **prop_zeroModulusThrows**: Подтверждено корректное выбрасывание исключения при попытке деления на ноль.
3. **treeDepthMax**: Проверена корректность рекурсивного обхода дерева.
4. **prop_addNodeNotDecrease**: Доказано, что добавление нового узла в любую позицию дерева не может уменьшить его общую глубину.

```

--- Testing isCongruent ---
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.

--- Testing treeDepth ---
+++ OK, passed 100 tests.
*** Failed! Falsified (after 1 test):
0
*** Failed! Falsified (after 1 test):
0
Empty
Empty
+++ OK, passed 100 tests.

```

Рис. 3: Вызов ошибок во второй части

При внесении преднамеренной ошибки в код (например, изменение формулы глубины на `max (depth 1) (depth r)`), QuickCheck успешно находит контрпример и выполняет операцию *shrinking* для поиска минимального ошибочного случая.

Заключение

В результате выполнения работы был разработан модульный проект на языке Haskell, реализующий функции математической логики и обработки древовидных структур данных. Использование библиотеки `QuickCheck` позволило облегчить процесс поиска ошибок и верифицировать свойства функций, включая граничные случаи (пустые деревья, отрицательные числа). Были освоены механизмы инкапсуляции в Haskell через экспорт модулей и методы тестирования исключений в функциональном коде.

Список литературы

- [1] Haskell Official Documentation – официальная документация по языку Haskell [Электронный ресурс]. <https://www.haskell.org/documentation/>
- [2] Миран Липовача. Изучай Haskell во имя добра! / Пер. с англ. — М.: ДМК Пресс, 2012. — 490 с.
- [3] QuickCheck: Automatic property-based testing of Haskell programs [Электронный ресурс]. <https://hackage.haskell.org/package/QuickCheck>
- [4] The Haskell Tool Stack User Guide [Электронный ресурс]. <https://docs.haskellstack.org/en/stable/README/>